With this quadratic mean function in hand, we can now write a GP model that initializes its mean module using the custom `QuadraticMean` class that we just implemented:

```
class QuadraticMeanGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = QuadraticMean()
        self.covar_module = gpytorch.kernels.RBFKernel()

    def forward(self, x):
        ...  (1)
```

(1) Omitted.

Rerunning our entire training procedure with gradent descent gives us the predictions in Figure 3.10.

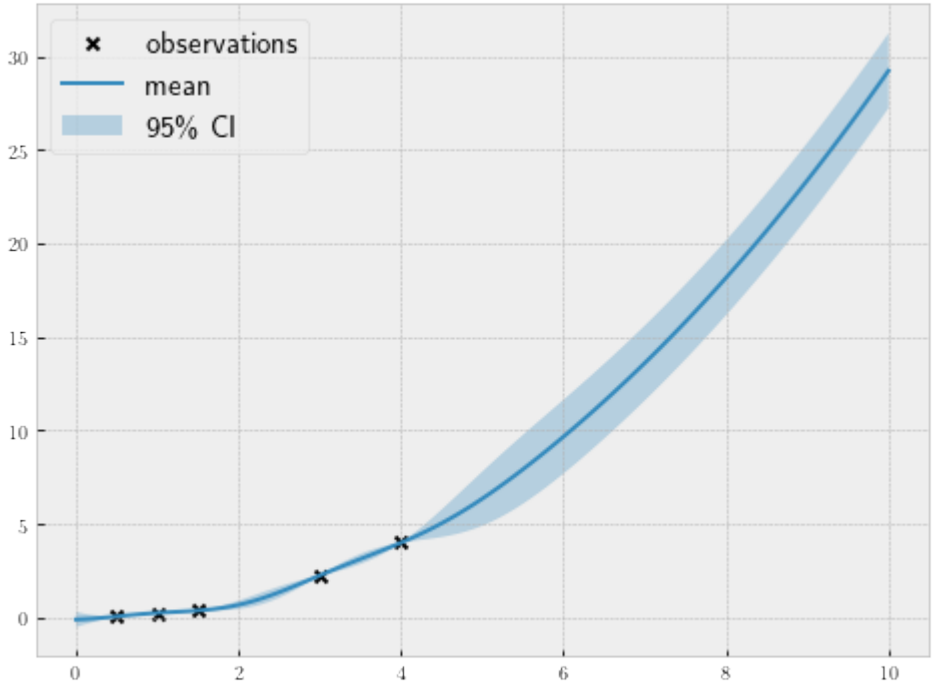


Figure 3.10 Predictions by a GP with a quadratic mean function. This GP predicts that with larger living area, the price increases at a faster rate.

Here, we successfully model the non-constant rate of increase in housing prices with respect to living area—our predictions increase much faster on the right side of the plot than on the left.

We can do this for any functional form that we want to assume about the objective, such as a higher-degree polynomial or a sublinear function. All we need to do is implement a mean function class with appropriate parameters, and use gradient descent to assign values to these parameters that will give us a good fit of our training data.

Our discussion so far demonstrates the mathematical flexibility of GP models, in that they can make use of a mean function of any structure and will still be able to produce probabilistic

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>